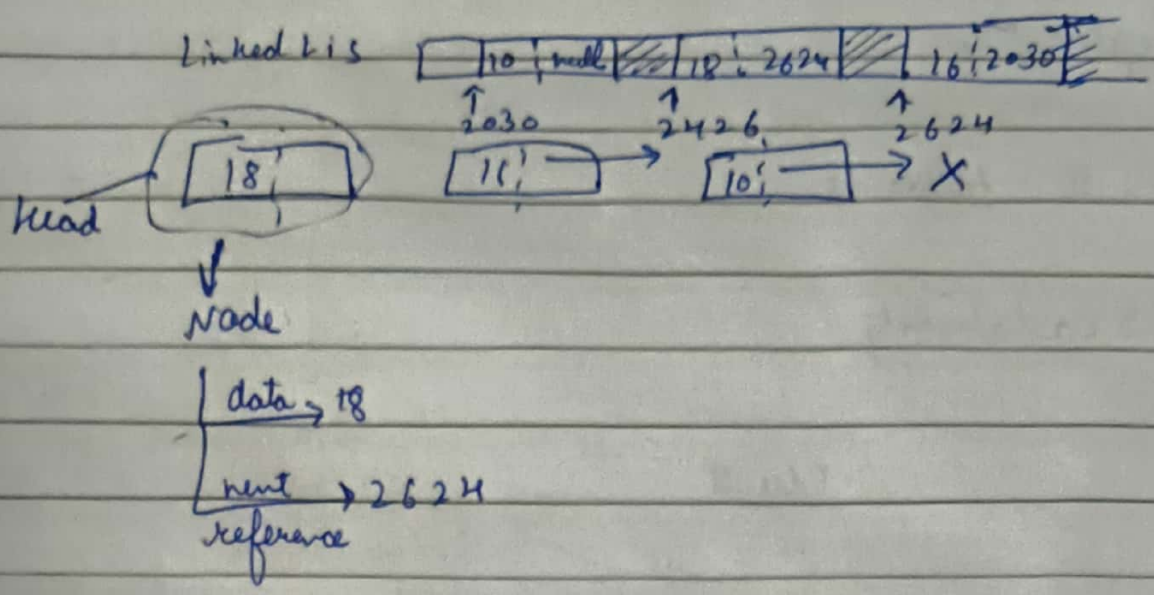
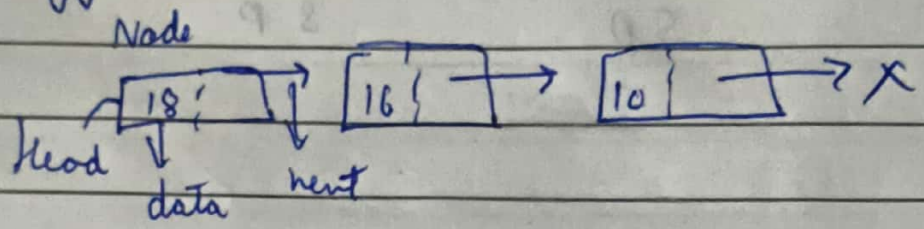
# Master in Linked List



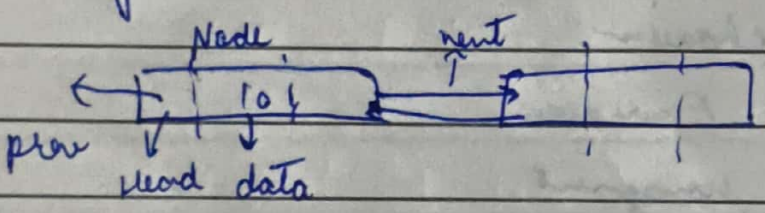

## 1. Singly-Linked List

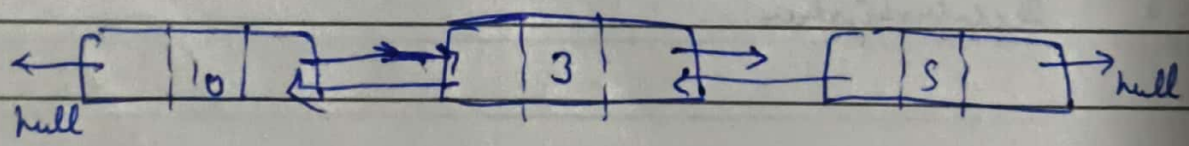



## 2. Doubly-linked list

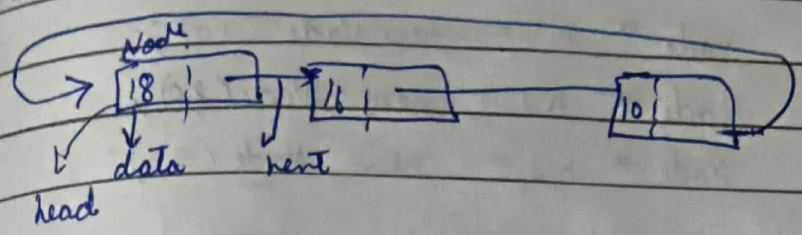
→ Each node has two pointer pard n such that p points to the previous node and n points to the next node.

Last node pointer to null



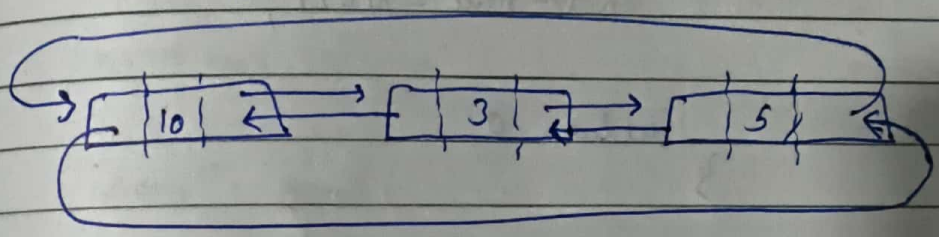



3. Circular - linked list :




Each node points to the next node and the last node points back to the first node

4. Circular - Doubly - linked list :




Time Complexity

Access : O(n)

Search : O(n)

Insert : O(1)

Remove : O(1)

Code :

```
class Node {
public:
    int data
    Node * next ;
    Node (int data) {
        this → data = data ;
        this → next = NULL ;
    }
};
```

```
int main() {
    Node * n1 = new Node (10);
    Node * n2 = new Node(20);
    Node * n3 = new Node (30);

    Node * head = n1;
    head -> next = n2;
    n2 -> next = n3;
    n3 -> next = NULL;

    return 0;
}
```

[10] [20] → [30]

Traverse in linked list

```
class Node {
public
    int data
    Node* next ;

    Node ( int data ) {
        this -> data = data;
        this -> next = NULL;

void traverse (Node* head) {
    Node * curr = head;
```

Node

| data | next |
|---|---|

```
while (curr != NULL) {
    cout << curr -> data << " ";
    curr = curr -> next;
}
}

int main () {
    Node* n1 = new Node (10);
    Node * n2 = new Node (20);

    n1 -> next = n2;

    Node* head = n1;
    traverse (head);
    return 0;
}
```

In C++ main must return integer

Node* head = n1;

or first element

[10] [20] [30] → X

curr print → 10

[10] [20] [30] → X

print → 10, 20 curr

[10] [20] [30] → X

curr

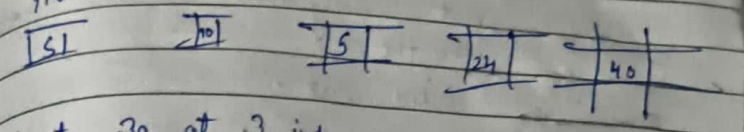
print → 10, 20, 30

To insert an element in [node] in the linkedlist

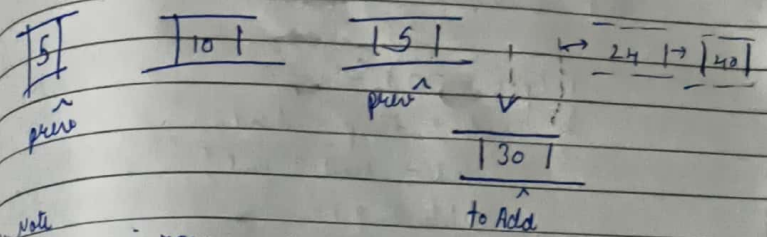
```
Node * ~~insert( int data, Node* head, int pos) {~~

Node * insert ( Node * head, int data, int pos) {

    Node * to_Add = new Node (data);

    if (pos == 0) {
        to Add -> next = head ;
        head => to Add;
        return head;
    }
    Node * prev = head;

    for (int i=0 ; i< pos-1 ; i++) {
        prev = prev -> next;
    }
    to Add -> next = prev -> next;
    prev -> next = toAdd;

    return head;
}
int main () {
    head = insert (head, 25, 2) ;
    return 0;
}
```

30 , head , 3) ;

Given




Insert 30 at 3 index




Note
Never miss your node reference ; otherwise you will lose your linked list

10 , 20 , 30

node * prev = head

for loop

prev = prev -> next  i<1

(i=0 ; i< 1-1 ; i++) {

} prev -> 20

to Add

10 , 20 , 30

let say at pos 3

10 , 20 , 30 to add

for (i=0 ; i< 3-1 ; i++)

{ prev => prev -> next

i=0  prev -> 20 -> 0

i=1  prev -> 30 ->

for (i=0; i< 2 ; i++)

{ prev = prev -> next

head = head → next

points to ~~next~~ memory address of the next node
Store the
in the linked list.

```
void deleteNode (int pos) {

    if (pos == 0) {
        Node * temp = head;
        head = head -> next;
        delete temp;
        return;
    }

    Node * prev = head;

    for (int i=0; i< pos-1; i++) {
        prev = prev -> next
    }

    Node * temp = prev -> next
    prev -> next = temp -> next;
    delete temp;
}
```

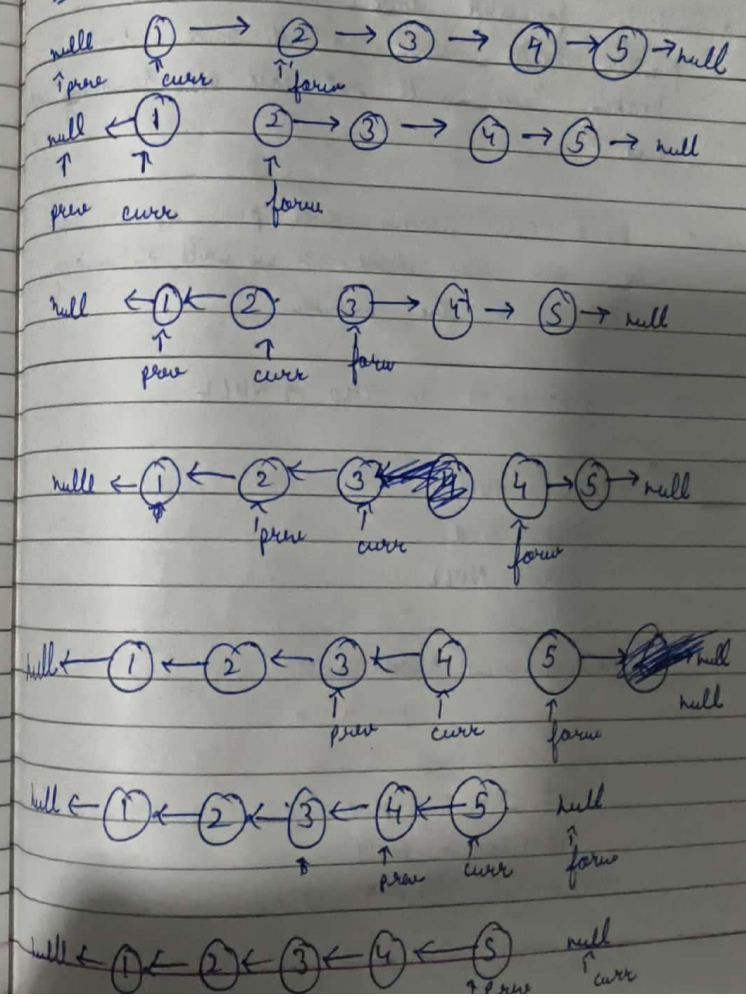
204. Reverse linked list

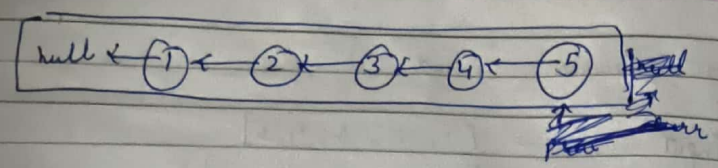
Input Head - [1, 2, 3, 4, 5]
output - [5, 4, 3, 2, 1]

To use 3 pointer → prev, curr ~~and~~ Forw.

~~null~~

⇒






- Prev will hold the previous value because, if we break the link. So we will not lose our linked list

- Similarly forward will point to the next pointer after curr. So that once link is broken we will not lose our remaining linked list.

- Once curr reaches null our prev will be our new head. So we will return our prev as the answer

Ex ① 10 → 20 → 30 → 40 → NULL

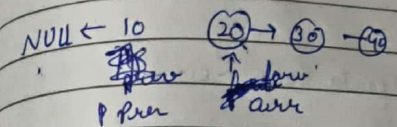
prev = NULL
curr = Head
Forw = NULL

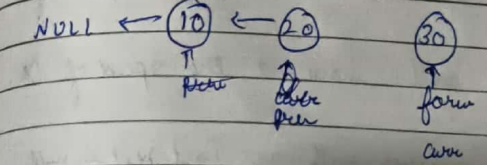
Forw = current → ~~next~~ next
Forw → 20

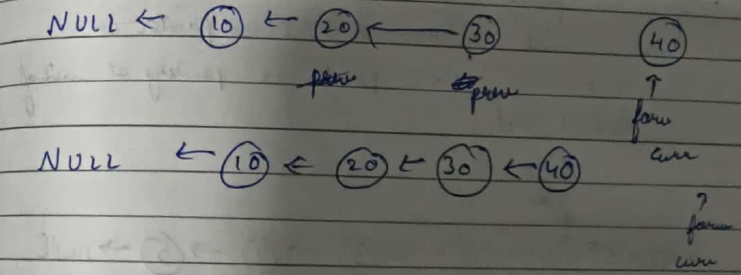
Current → next = prev
10 → next = NULL
~~NULL ←~~ 10 10 → NULL

prev = ~~prev~~ curr
prev → 10

curr = forw
curr → 20



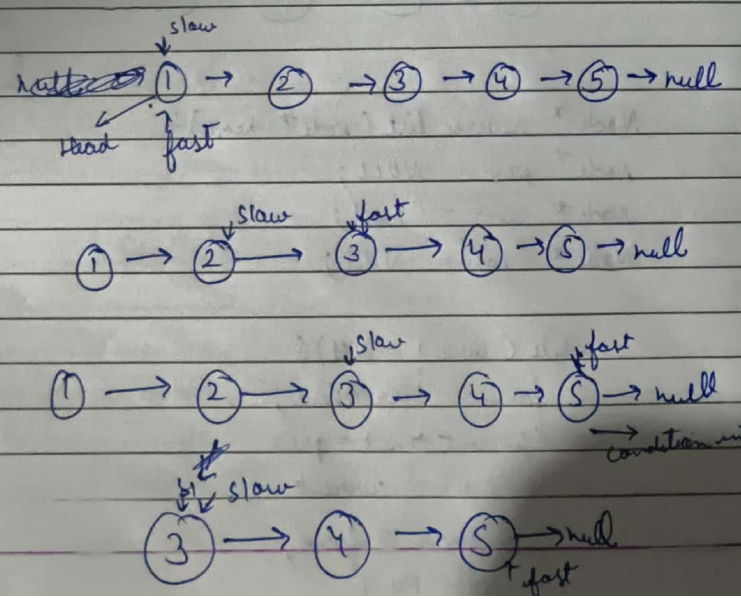

```
Node* reverse list (Node* head) {
Node* prev = NULL;
Node* curr = head;
Node* forw = NULL;

while (curr != NULL) {
forw = curr → next;
curr → next = prev
prev = curr
curr = forw;
```

876. Middle of the linked list

head = [1, 2, 3, 4, 5]
output [3, 4, 5]

We will create 2 pointer faster & slow

- Fast pointer will move at the speed of 2x
- Slow pointer will move at the speed of 1x.
- If fast pointer reaches null or fast.next is null we will return our slow pointer which means our slow pointer is pointing at mid of linked list.



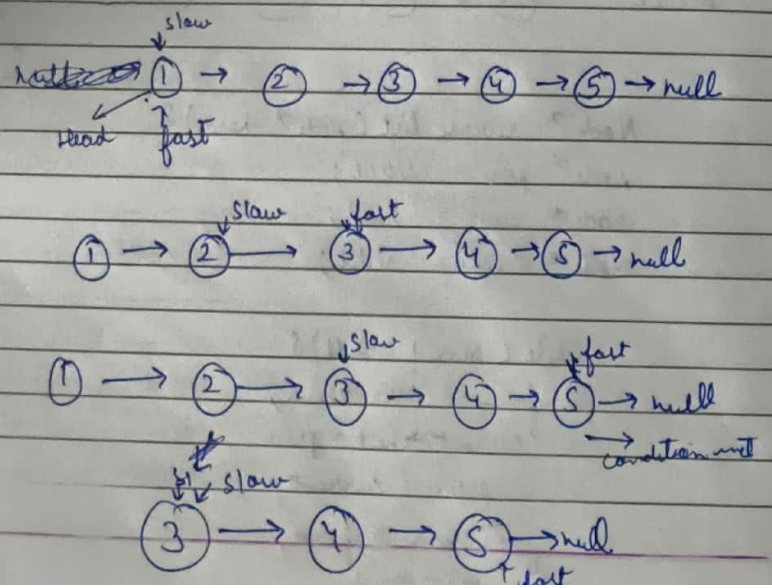

```
List Node * middleNode (List Node * head)
    if (head -> next == NULL)
        return head;
    }

    List Node * slow = head;
    List Node * fast = head;

    while (fast != NULL && fast -> next != NULL) {
        fast = fast -> next -> next;
        slow = slow -> next;
    }

    return slow;
```

876. Middle of the Linked List

head = [1, 2, 3, 4, 5]
output [3, 4, 5]

We will create 2 pointer faster & slower

- fast pointer will move at the speed of 2x
- Slow pointer will move at the speed of 1x.
- If fast pointer reaches null or fast.next is null we will return our slow pointer which means our slow pointer is pointing at mid of linked list.




```
List Node * middleNode (List Node * head)
    if (head -> next == NULL) {
        return head;
    }

    List Node * slow = head;
    List Node * fast = head;

    while (fast != NULL && fast -> next != NULL) {
        fast = fast -> next -> next;
        slow = slow -> next;
    }

    return slow;
```